ĐẠI HỌC CẦN THƠ
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



BÁO CÁO ĐỒ ÁN MÔN HỌC

TIN HỌC LÝ THUYẾT

ĐỀ TÀI:

WEBSITE TƯƠNG TÁC GIẢI BÀI TẬP AUTOMATA

Sinh viên thực hiện:
Họ và tên: Huỳnh Thanh Phong
MSSV: B2207555
Số thứ tự: 10

Giảng viên giảng dạy:
GVHD.ThS. ***Phạm Xuân Hiền***

Cần Thơ, ngày **30** tháng **10** năm **2025**

ĐẠI HỌC CẦN THƠ
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



BÁO CÁO ĐỒ ÁN MÔN HỌC

TIN HỌC LÝ THUYẾT

ĐỀ TÀI:

WEBSITE TƯƠNG TÁC GIẢI BÀI TẬP AUTOMATA

Sinh viên thực hiện:
Họ và tên: Huỳnh Thanh Phong
MSSV: B2207555
Số thứ tự: 10

Giảng viên giảng dạy:
GVHD.ThS. ***Phạm Xuân Hiền***

Cần Thơ, ngày **30** tháng **10** năm **2025**

LỜI CẢM ƠN

Trong suốt quá trình học tập và thực hiện đồ án môn học Tin học lý thuyết, em đã có cơ hội được tiếp cận, tìm hiểu và vận dụng các kiến thức đã học vào việc nghiên cứu và triển khai đề tài "Website giải bài tập hỗ trợ học tập" Đây là một trải nghiệm quý giá, giúp chúng em không chỉ củng cố nền tảng lý thuyết mà còn rèn luyện kỹ năng tư duy, nghiên cứu và làm việc nhóm.

Để có thể hoàn thành tốt báo cáo này, nhóm xin bày tỏ lòng biết ơn sâu sắc đến Giảng viên ThS. Phạm Xuân Hiền, người đã tận tình hướng dẫn, chỉ bảo và hỗ trợ chúng em trong suốt quá trình thực hiện đồ án. Cô không chỉ giúp chúng em hiểu rõ hơn về các nguyên lý và kỹ thuật trong lĩnh vực mã hóa và bảo mật thông tin, mà còn truyền đạt cho chúng em tinh thần nghiêm túc trong học tập và nghiên cứu khoa học. Những nhận xét và góp ý quý báu của cô đã giúp nhóm hoàn thiện đề tài một cách hệ thống và chặt chẽ hơn.

Bên cạnh đó, sự khích lệ và định hướng của cô đã giúp nhóm vượt qua những khó khăn, vướng mắc trong quá trình tìm hiểu và cài đặt thuật toán. Những kiến thức và kinh nghiệm mà cô truyền đạt sẽ là hành trang quý giá giúp chúng em tự tin hơn trong con đường học tập và làm việc sau này.

Một lần nữa, nhóm xin chân thành cảm ơn cô Phạm Xuân Hiền vì sự tận tâm, trách nhiệm và nhiệt huyết trong giảng dạy. Nhóm xin kính chúc cô luôn dồi dào sức khỏe, hạnh phúc và tiếp tục gặt hái nhiều thành công trong sự nghiệp trồng người cao quý.

Cần Thơ, ngày 30 tháng 10 năm 2025

Sinh viên thực hiện

Huỳnh Thanh Phong

TÓM TẮT

Trong thời đại công nghệ số phát triển mạnh mẽ, việc **kiểm tra và phân tích các mô hình tự động hữu hạn** như DFA, NFA và NFA- ngày càng trở nên quan trọng trong giáo dục, lý thuyết tính toán và ứng dụng phần mềm. Website **DFA-NFA Checker** được xây dựng nhằm cung cấp một công cụ trực tuyến **nhanh chóng, trực quan và dễ sử dụng** cho việc kiểm tra chuỗi đầu vào, đánh giá sự chấp nhận của các tự động, và hỗ trợ chuyển đổi giữa các dạng tự động: NFA sang DFA, NFA- sang NFA hoặc DFA.

- **Kiểm tra chuỗi:** Người dùng nhập một chuỗi ký tự, hệ thống sẽ hiển thị từng bước chuyển trạng thái của DFA hoặc NFA, làm nổi bật trạng thái hiện tại, trạng thái bắt đầu và trạng thái kết thúc.
- **Chuyển đổi tự động:** Hỗ trợ thuật toán *subset construction* và loại bỏ -transition, giúp chuyển đổi NFA/NFA- sang DFA/NFA một cách trực quan và dễ hiểu.
- **Minh họa đồ thị:** Tự động vẽ đồ thị trạng thái với màu sắc trực quan: trạng thái bắt đầu màu xanh lá, trạng thái kết thúc màu đỏ, các trạng thái còn lại màu xanh dương, giúp người học và lập trình viên dễ quan sát và phân tích.
- **Ứng dụng thực tế:** Công cụ này phục vụ mục đích giáo dục, kiểm thử bài tập lý thuyết tự động, và hỗ trợ phát triển các hệ thống xử lý ngôn ngữ, giao thức truyền thông, hoặc phần mềm yêu cầu mô hình hóa trạng thái.

Hệ thống được cài đặt với giao diện web thân thiện, kết hợp các thuật toán lý thuyết và trực quan hóa đồ thị, **tăng cường trải nghiệm học tập và nghiên cứu**, đồng thời giúp người dùng nhanh chóng kiểm tra và chuyển đổi tự động hữu hạn một cách hiệu quả.

ABSTRACT

In the era of rapid digital technology development, the verification and analysis of **finite automata models** such as DFA, NFA, and NFA- have become increasingly important in education, computational theory, and software applications. The **DFA-NFA Checker** website provides a **fast, visual, and user-friendly** online tool for string validation, automaton acceptance testing, and conversion between automata types: NFA to DFA, NFA- to NFA or DFA.

- **String checking:** Users input a string, and the system displays each step of state transitions in DFA or NFA, highlighting current, start, and accepting states.
- **Automata conversion:** Supports *subset construction* and -transition elimination algorithms to convert NFA/NFA- to DFA/NFA clearly and intuitively.
- **Graph visualization:** Automatically draws state diagrams with intuitive colors: start state in green, accepting states in red, and others in blue, aiding learners and developers in analysis.
- **Practical application:** Serves educational purposes, automaton homework verification, and supports development of language processing systems, communication protocols, or software requiring state modeling.

Implemented as a web-based system with an intuitive interface, combining theoretical algorithms and graphical visualization, the tool **enhances learning and research experience**, allowing users to efficiently validate and convert finite automata.

MỤC LỤC

DANH MỤC HÌNH **1**

1 MÔ TẢ BÀI TOÁN **2**

2 MỤC TIÊU BÀI TOÁN **4**
2.1 Mục tiêu chính 4
2.2 Phạm vi và chức năng hệ thống 4
2.3 Công nghệ sử dụng 5
2.4 Ý nghĩa thực tiễn 5

3 THIẾT KẾ VÀ CÀI ĐẶT HỆ THỐNG **6**
3.1 Thiết kế hệ thống 6
3.2 Thiết kế giao diện 6
3.3 Thiết kế dữ liệu 7
3.4 Thuật toán chính 7
3.5 Cài đặt hệ thống 8

4 CƠ SỞ LÝ THUYẾT **9**
4.1 Định nghĩa và ký hiệu cơ bản 9
4.1.1 Deterministic Finite Automaton (DFA) 9
4.1.2 Non-Deterministic Finite Automaton (NFA) 10
4.1.3 Non-Deterministic Finite Automaton với epsilon (NFAϵ) 11
4.2 Chuyển đổi NFA sang DFA 12
4.3 Chuyển NFAϵ sang NFA hoặc DFA 13
4.3.1 Epsilon-closure (E-closure) 14
4.3.2 Thuật toán chuyển NFAϵ sang NFA 14
4.3.3 Thuật toán chuyển NFAϵ sang DFA 14
4.4 Kiểm tra chuỗi trong DFA, NFA, NFAϵ 15

5 CHỨC NĂNG CỦA HỆ THỐNG **16**
5.1 Tổng quan 16
5.2 Chức năng kiểm tra DFA 16
5.3 Chức năng kiểm tra NFA 18
5.4 Chức năng kiểm tra NFAϵ 21
5.5 Chức năng chuyển NFA sang DFA 23
5.6 Chức năng chuyển NFAϵ sang DFA hoặc NFA 25
5.7 Chức năng lý thuyết và bài tập 28

6 TỔNG KẾT VÀ ĐÁNH GIÁ HƯỚNG PHÁT TRIỂN **30**
6.1 Ưu điểm của hệ thống 30
6.2 Hạn chế của hệ thống 30
6.3 Hướng phát triển 31

DANH MỤC HÌNH

1.1 Sơ đồ minh họa NFA, DFA và NFAe với các chuyển đổi đứt nét 2

2.1 Công nghệ thiết kết website 5

3.1 Sơ đồ mô tả dữ liệu ôtômát dạng JSON 7
3.2 Sơ đồ quy trình vận hành hệ thống 8
3.3 Ôtômát 8 trạng thái với mã màu: start (xanh lá), normal (xanh dương), accept (đỏ). 8

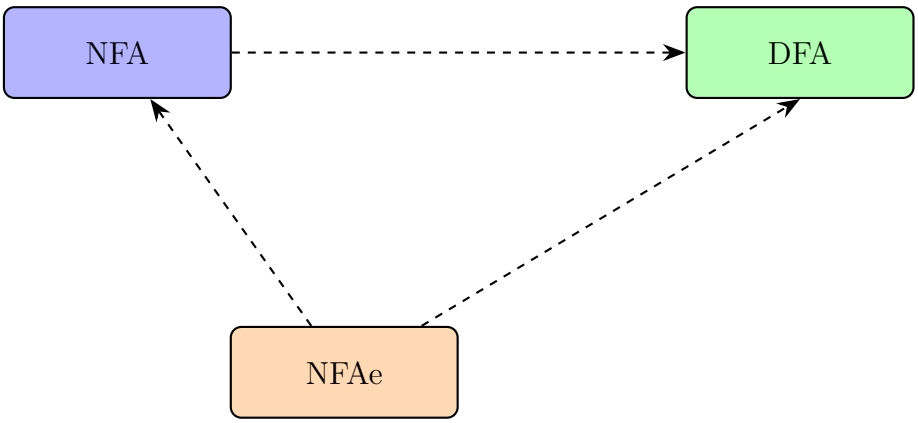
5.1 Giao diện chính của website 16
5.2 Giao diện trang kiểm tra DFA 17
5.3 Hàm simulateDFA kiểm tra chuỗi thõa mãn DFA 18
5.4 Giao diện trang kiểm tra NFA 19
5.5 Hàm simulateNFA kiểm tra chuỗi thõa mãn NFA 20
5.6 Giao diện trang kiểm tra NFAe 21
5.7 Hàm simulateNFAe kiểm tra chuỗi thõa mãn NFAe 22
5.8 Giao diện chức năng chuyển NFA sang DFA 23
5.9 Hàm convertNFAtoDFA chuyển NFA sang DFA 24
5.10 Giao diện chức năng chuyển NFAe sang NFA hoặc DFA 25
5.11 Hàm epsilonClosure 26
5.12 Hàm convertNFAeToNFA chuyển NFAe sang NFA 27
5.13 Hàm convertNFAtoDFA chuyển NFAe sang DFA 28

Chương 1

MÔ TẢ BÀI TOÁN

Trong các chương trình học môn Tin học lý thuyết, đặc biệt học phần *Lý thuyết ôtômát và ngôn ngữ hình thức*, sinh viên thường phải tiếp cận những khái niệm trừu tượng như **Máy hữu hạn xác định (DFA – Deterministic Finite Automaton)**, **Máy hữu hạn không xác định (NFA – Non-Deterministic Finite Automaton)** và **NFA có chuyển ϵ (NFAe)**. Các mô hình này là nền tảng trong việc nghiên cứu về ngôn ngữ hình thức, các phép chuyển đổi, cũng như kiểm tra chuỗi đầu vào có được chấp nhận bởi một máy hay không. Tuy nhiên, do đặc tính rất hình thức và trừu tượng của chúng, nhiều sinh viên gặp khó khăn trong việc hình dung cơ chế hoạt động, hiểu cách các trạng thái chuyển đổi, cũng như sự khác biệt giữa các loại máy (DFA vs. NFA vs. -NFA), và nhất là trong việc chuyển đổi giữa các dạng (ví dụ từ NFA sang DFA) hoặc kiểm thử bằng dữ liệu thực.

Khi dạy học truyền thống, phương pháp chủ yếu là giảng bài lý thuyết kèm theo vẽ đồ thị máy lên bảng hoặc yêu cầu sinh viên tự vẽ tay và phân tích. Chính vì tính "khô khan" và mang nặng hình thức, quá trình học và làm bài tập trở nên khó khăn và đôi khi thiếu động lực. Nhiều nghiên cứu quốc tế đã chỉ ra rằng việc sử dụng các công cụ trực quan, tương tác giúp nâng cao việc hiểu và ghi nhớ các khái niệm về ôtômát. Ví dụ, trong bài báo *"A Web App for Teaching Finite State Automata"* các tác giả giới thiệu một công cụ trực tuyến cho phép sinh viên tạo, mô phỏng và xuất các máy hữu hạn, và nhận được phản hồi tích cực rằng điều này cải thiện việc học [2]. Hay như nghiên cứu *"Migrating teaching of automata theory to a digital platform"* cho thấy rằng phương pháp số hóa giúp tăng khả năng tiếp thu và tham gia của sinh viên trong phần lý thuyết ôtômát [1].



Hình 1.1: Sơ đồ minh họa NFA, DFA và NFAe với các chuyển đổi đứt nét

Từ thực trạng đó, đề tài *"Website hỗ trợ học tập và giải bài tập môn Tin học lý thuyết"* được kiến nghị nhằm đáp ứng nhu cầu thiết thực: xây dựng một công cụ trực

tuyến tương tác, cho phép sinh viên **kiểm tra**, **mô phỏng**, **chuyển đổi** giữa các mô hình ôtômát (DFA, NFA, -NFA) một cách trực quan, sinh động và dễ hiểu. Công cụ này sẽ hỗ trợ sinh viên trong việc:

- Hình dung cấu trúc và cơ chế hoạt động của mỗi loại ôtômát (các trạng thái, các chuyển tiếp, trạng thái bắt đầu và trạng thái chấp nhận).
- Kiểm thử các chuỗi đầu vào và quan sát quá trình chuyển trạng thái theo từng bước, giúp sinh viên hiểu rõ hơn tại sao một chuỗi được chấp nhận hoặc bị từ chối.
- Thực hiện chuyển đổi giữa các dạng ôtômát (ví dụ từ -NFA sang NFA, hoặc từ NFA sang DFA) và quan sát kết quả, từ đó củng cố khả năng tư duy trừu tượng và kỹ năng giải bài tập.
- Giải bài tập tự động hoặc bán tự động, hỗ trợ sinh viên luyện tập nhiều lần, nhận phản hồi tức thì và từ đó nâng cao hiệu quả học tập.

Kết hợp giữa yếu tố trực quan hóa, mô phỏng và tương tác số hóa, đề tài hướng tới việc **giảm bớt tính khô khan, trừu tượng** của phần học lý thuyết này và đồng thời **tăng hứng thú, cải thiện hiệu quả học tập** cho sinh viên. Qua đó, không chỉ giúp sinh viên nắm chắc các khái niệm ôtômát mà còn rèn luyện kỹ năng phân tích, chuyển đổi mô hình và giải bài tập — yếu tố then chốt giúp họ thành thạo môn học này và có nền tảng vững chắc cho các môn tiếp theo về lý thuyết tính toán và ngôn ngữ lập trình.

Chương 2

MỤC TIÊU BÀI TOÁN

2.1 Mục tiêu chính

Hệ thống website được xây dựng với các mục tiêu cơ bản sau:

- Hỗ trợ sinh viên kiểm tra chuỗi có được chấp nhận bởi một ôtômát hữu hạn (DFA, NFA hoặc NFAϵ) hay không.
- Cho phép chuyển đổi tự động giữa các mô hình ôtômát:
  - NFA $\rightarrow$ DFA theo thuật toán xây dựng tập con (subset construction).
  - NFAϵ $\rightarrow$ NFA thông qua epsilon-closure.
  - NFAϵ $\rightarrow$ DFA thông qua kết hợp hai bước trên.
- Cung cấp giao diện trực quan, thể hiện các trạng thái và chuyển trạng thái của ôtômát dưới dạng đồ thị, giúp người học dễ dàng quan sát quá trình hoạt động.
- Hỗ trợ người dùng nhập dữ liệu ôtômát thủ công hoặc từ file, đồng thời kiểm tra chuỗi đầu vào theo từng bước.
- Giúp sinh viên rèn luyện kỹ năng giải bài tập ôtômát, củng cố lý thuyết thông qua thao tác thực hành trực tiếp.

2.2 Phạm vi và chức năng hệ thống

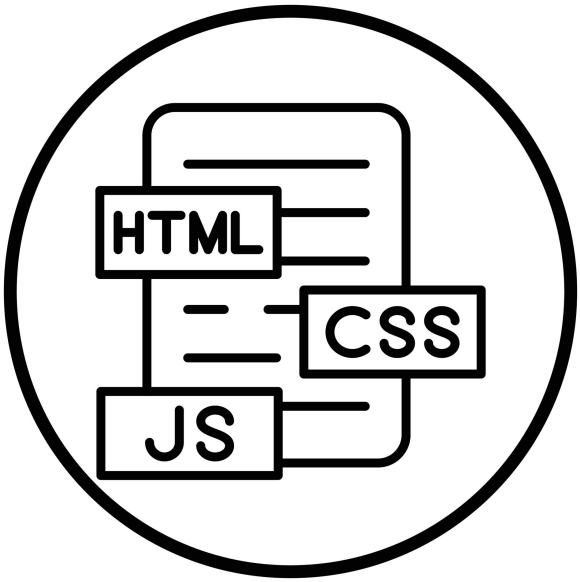
Hệ thống website tập trung vào các chức năng chính sau:

1. **Nhập dữ liệu ôtômát:** cho phép sinh viên nhập danh sách trạng thái, bảng chuyển, ký hiệu đầu vào, trạng thái bắt đầu và trạng thái kết thúc, hoặc tải/lưu ôtômát dưới dạng file `.txt`.
2. **Kiểm tra chuỗi:** cho phép nhập một chuỗi bất kỳ và kiểm tra xem chuỗi đó có được chấp nhận hay không. Quá trình kiểm tra được hiển thị từng bước chuyển, giúp sinh viên quan sát trực tiếp hoạt động của ôtômát.
3. **Chuyển đổi giữa các mô hình:** hỗ trợ NFA $\rightarrow$ DFA, NFAϵ $\rightarrow$ NFA và NFAϵ $\rightarrow$ DFA.
4. **Hiển thị trực quan:** sử dụng đồ thị trạng thái, phân biệt trạng thái bắt đầu, kết thúc bằng màu sắc khác nhau, làm nổi bật các bước chuyển trong quá trình kiểm tra chuỗi.

5. **Tính năng học tập mở rộng:** cung cấp bài tập ngẫu nhiên, so sánh kết quả tự giải với đáp án chuẩn, và gợi ý quy tắc chuyển đổi hoặc thuật toán lý thuyết.

2.3 Công nghệ sử dụng

- HTML5, CSS3, JavaScript, Bootstrap, thư viện vẽ đồ thị `vis_network.js`.



Hình 2.1: Công nghệ thiết kết website

- **Thuật toán:** Kiểm tra chuỗi DFA/NFA/NFAϵ, chuyển đổi NFA $\rightarrow$ DFA, NFAϵ $\rightarrow$ NFA, NFAϵ $\rightarrow$ DFA, epsilon-closure, xây dựng tập con.

2.4 Ý nghĩa thực tiễn

Hệ thống giúp sinh viên củng cố kiến thức về ôtômát, áp dụng công nghệ web vào giảng dạy lý thuyết Tin học, tạo môi trường học tập trực quan, sinh động và dễ tiếp cận. Giảng viên có thể sử dụng để minh họa bài giảng; sinh viên có thể tự học, thực hành và kiểm chứng kết quả mà không cần phần mềm cài đặt phức tạp. Đây là công cụ hữu ích góp phần nâng cao hiệu quả tiếp thu môn học.

Chương 3

THIẾT KẾ VÀ CÀI ĐẶT HỆ THỐNG

3.1 Thiết kế hệ thống

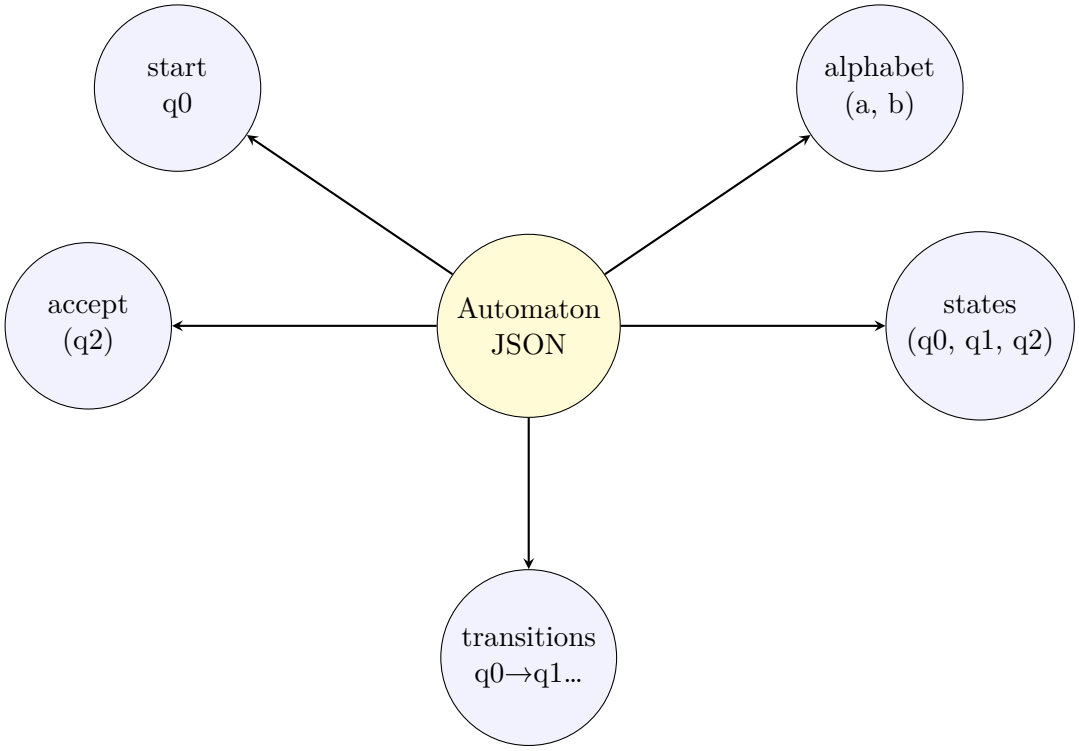
Hệ thống được thiết kế theo mô hình **client-server** với frontend đảm nhiệm giao diện người dùng và backend xử lý thuật toán ôtômát. Kiến trúc hệ thống bao gồm:

- **Frontend:** hiển thị đồ thị ôtômát, cung cấp giao diện nhập dữ liệu và kiểm tra chuỗi, sử dụng `vis_network.js` để trực quan hóa.
- **Backend:** xử lý thuật toán kiểm tra chuỗi, chuyển đổi NFA/NFAϵ sang DFA, tính epsilon-closure, xây dựng tập con.
- **Lưu trữ dữ liệu:** LocalStorage dùng để lưu trữ ôtômát người dùng đã nhập, cho phép tải lại khi cần.

3.2 Thiết kế giao diện

- **Trang nhập ôtômát:** form nhập trạng thái, ký hiệu đầu vào, bảng chuyển.
- **Trang kiểm tra chuỗi:** nhập chuỗi đầu vào, hiển thị các bước chuyển trạng thái, tô màu trạng thái đang được xử lý.
- **Trang chuyển đổi ôtômát:** chọn loại chuyển đổi, hiển thị kết quả đồ thị sau chuyển đổi.
- **Trang bài tập luyện tập:** sinh viên có thể thử nghiệm với bài tập ngẫu nhiên và so sánh đáp án.

3.3 Thiết kế dữ liệu



Hình 3.1: Sơ đồ mô tả dữ liệu ôtômát dạng JSON

3.4 Thuật toán chính

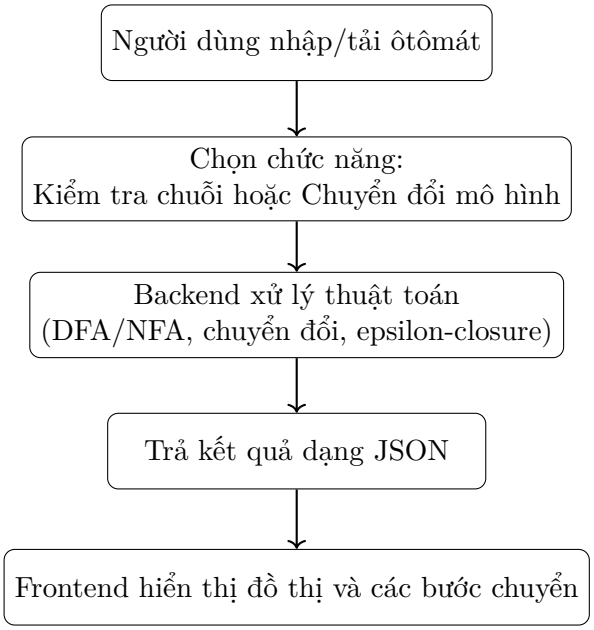
1. Kiểm tra chuỗi Chức năng cho phép người dùng nhập một chuỗi để xác định xem chuỗi có được chấp nhận bởi ôtômát (DFA hoặc NFA) hay không. Hệ thống hiển thị từng bước chuyển trạng thái và làm nổi bật trạng thái hiện tại trên sơ đồ, giúp người học theo dõi trực quan quá trình duyệt.

2. Chuyển đổi NFA → DFA Hệ thống sử dụng thuật toán *subset construction* để chuyển một NFA sang DFA tương đương. Mỗi trạng thái của DFA được biểu diễn như một tập các trạng thái NFA. Kết quả được xuất ra dưới dạng JSON và sơ đồ DFA với nhãn dạng {q0, q1}.

3. Chuyển đổi NFAϵ → NFA Việc loại bỏ các chuyển ϵ được thực hiện bằng thuật toán *epsilon-closure*. Với mỗi trạng thái, hệ thống tính tập các trạng thái có thể đạt được thông qua một hoặc nhiều bước ϵ, sau đó thay thế các cạnh ϵ bằng các cạnh tương ứng. Kết quả là một NFA tương đương không còn chuyển ϵ.

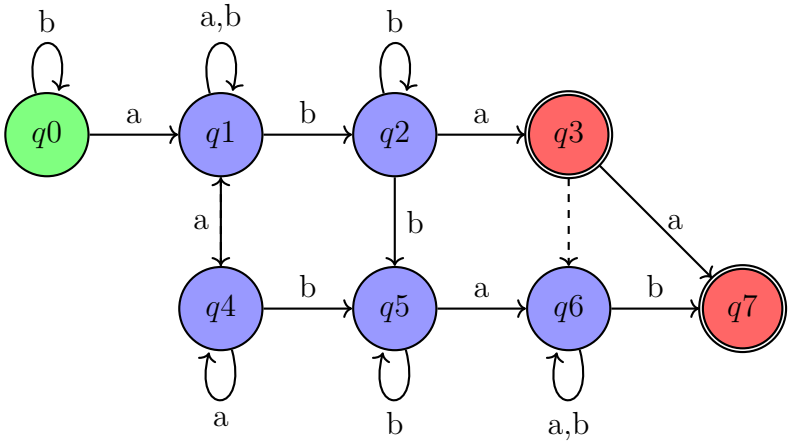
4. Chuyển đổi NFAϵ → DFA Quá trình gồm hai bước: (1) loại bỏ các chuyển ϵ bằng epsilon-closure để thu được NFA không ϵ; (2) áp dụng *subset construction* để tạo ra DFA tương đương. Hệ thống thực hiện tự động và trả về DFA cuối cùng dưới dạng sơ đồ và JSON.

3.5 Cài đặt hệ thống



Hình 3.2: Sơ đồ quy trình vận hành hệ thống

Hệ thống biểu diễn ôtômát dưới dạng đồ thị trạng thái, trong đó mỗi trạng thái được hiển thị bằng một nút có màu sắc phân biệt: trạng thái bắt đầu được tô xanh lá, các trạng thái kết thúc được tô đỏ, và các trạng thái thông thường được tô xanh dương. Các chuyển trạng thái được vẽ dưới dạng mũi tên hướng, thể hiện trực quan cách ôtômát xử lý chuỗi đầu vào.



Hình 3.3: Ôtômát 8 trạng thái với mã màu: start (xanh lá), normal (xanh dương), accept (đỏ).

Chương 4

CƠ SỞ LÝ THUYẾT

4.1 Định nghĩa và ký hiệu cơ bản

4.1.1 Deterministic Finite Automaton (DFA)

Định nghĩa: Một **DFA** là một ôtômát hữu hạn xác định, được định nghĩa bởi 5 thành phần:

$$DFA = (Q, \Sigma, \delta, q_0, F)$$

trong đó:

- Q là tập hữu hạn các trạng thái.
- Σ là tập ký hiệu đầu vào (alphabet).
- $\delta : Q \times \Sigma \to Q$ là hàm chuyển trạng thái xác định.
- $q_0 \in Q$ là trạng thái bắt đầu.
- $F \subseteq Q$ là tập các trạng thái kết thúc (accepting states).

Nguyên tắc hoạt động:

- Với mỗi ký hiệu đầu vào $a \in \Sigma$, từ trạng thái q sẽ chuyển sang đúng một trạng thái tiếp theo $\delta(q, a)$.
- Chuỗi $w = a_1 a_2 \dots a_n \in \Sigma^*$ được chấp nhận nếu hàm mở rộng $\hat{\delta}(q_0, w) \in F$.

Hàm chuyển mở rộng:

$$\hat{\delta} : Q \times \Sigma^* \to Q$$

được định nghĩa đệ quy:

$$\hat{\delta}(q, \epsilon) = q$$

$$\hat{\delta}(q, xa) = \delta(\hat{\delta}(q, x), a), \quad x \in \Sigma^*, a \in \Sigma$$

Ngôn ngữ được chấp nhận: Ngôn ngữ của DFA M được định nghĩa là tập tất cả các chuỗi mà DFA chấp nhận:

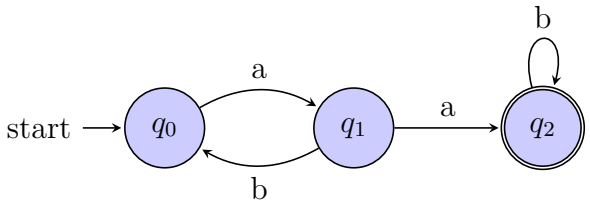
$$L(M) = \{w \in \Sigma^* \mid \hat{\delta}(q_0, w) \in F\}.$$

Tính chất quan trọng của DFA:

- Với mỗi trạng thái và ký tự đầu vào, có **chính xác một** trạng thái tiếp theo.

- Ngôn ngữ chấp nhận bởi DFA luôn là **ngôn ngữ chính quy (regular language)**.
- DFA có thể được tối tiểu hóa thành DFA tối tiểu duy nhất (đến đẳng cấu) bằng thuật toán Hopcroft.
- Ngôn ngữ chính quy đóng với các phép toán: hợp, giao, bù, hiệu, nối (concatenation), lặp Kleene ($*$).

Ví dụ minh họa: nhận các chuỗi trên $\{a, b\}$ mà có chữ a xuất hiện một lần hoặc hai lần liên tiếp cuối cùng:



Chạy DFA: Ví dụ với chuỗi $w = aab$:

$$\hat{\delta}(q_0, a) = q_1, \quad \hat{\delta}(q_1, a) = q_2, \quad \hat{\delta}(q_2, b) = q_2 \in F$$

$\rightarrow$ Chuỗi được chấp nhận.

Thuật toán kiểm tra chuỗi:

1. Khởi tạo trạng thái hiện tại $q = q_0$.
2. Duyệt từng ký tự a_i trong chuỗi w:
$$q \leftarrow \delta(q, a_i)$$
3. Nếu $q \in F$ sau khi hết chuỗi $\rightarrow$ chuỗi được chấp nhận.

Tối tiểu hóa DFA: - Loại bỏ các trạng thái không thể tới được (unreachable). - Gom các trạng thái tương đương (cùng dẫn tới chấp nhận/không chấp nhận giống nhau). - Thuật toán Hopcroft: phân hoạch trạng thái thành các lớp tương đương, tạo DFA tối tiểu. Bên trái là DFA ban đầu, có thể có các trạng thái không cần thiết hoặc tương đương. Bên phải là DFA tối tiểu sau khi loại bỏ trạng thái không thể tới và gom các trạng thái tương đương (Hopcroft).

4.1.2 Non-Deterministic Finite Automaton (NFA)

Định nghĩa: Một **NFA** là ôtômát hữu hạn không xác định, được định nghĩa bởi 5 thành phần:

$$NFA = (Q, \Sigma, \delta, q_0, F)$$

trong đó:

- Q là tập hữu hạn các trạng thái.
- Σ là tập ký hiệu đầu vào (alphabet).
- $\delta : Q \times \Sigma \rightarrow P(Q)$ là hàm chuyển trạng thái, mỗi cặp (q, a) có thể dẫn tới nhiều trạng thái.

- $q_0 \in Q$ là trạng thái bắt đầu.
- $F \subseteq Q$ là tập các trạng thái kết thúc (accepting states).

Nguyên tắc hoạt động:

- Tại một trạng thái q và ký tự đầu vào $a \in \Sigma$, NFA có thể chuyển sang **một hoặc nhiều trạng thái** trong $\delta(q, a)$.
- Chuỗi $w = a_1 a_2 \dots a_n$ được chấp nhận nếu tồn tại ít nhất một chuỗi trạng thái:

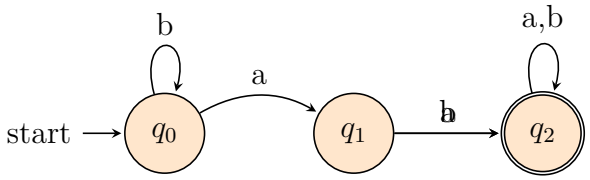
$$q_0 \xrightarrow{a_1} q_1 \xrightarrow{a_2} \dots \xrightarrow{a_n} q_n \in F$$

Hàm mở rộng: Hàm mở rộng $\hat{\delta} : Q \times \Sigma^* \to P(Q)$ được định nghĩa bằng:

$$\hat{\delta}(q, \epsilon) = \{q\}$$

$$\hat{\delta}(q, xa) = \bigcup_{p \in \hat{\delta}(q,x)} \delta(p, a), \quad x \in \Sigma^*, a \in \Sigma$$

Ví dụ minh họa NFA: nhận các chuỗi trên $\{a, b\}$ có chữ a xuất hiện bất kỳ vị trí nào nhưng chỉ một lần cuối cùng.



Ghi chú:

- Mỗi ký tự đầu vào có thể dẫn tới nhiều trạng thái.
- NFA có thể chấp nhận một chuỗi nếu tồn tại ít nhất một đường đi từ trạng thái bắt đầu đến trạng thái kết thúc.
- NFA luôn có thể được chuyển đổi sang DFA bằng thuật toán **subset construction**.

4.1.3 Non-Deterministic Finite Automaton với epsilon (NFAϵ)

Định nghĩa: Một **NFA**ϵ là ôtômát hữu hạn không xác định, cho phép chuyển trạng thái mà không cần đọc ký hiệu đầu vào (chuyển ϵ), được định nghĩa bởi 5 thành phần:

$$NFA\epsilon = (Q, \Sigma, \delta, q_0, F)$$

trong đó:

- Q là tập hữu hạn các trạng thái.
- Σ là tập ký hiệu đầu vào (alphabet).
- $\delta : Q \times (\Sigma \cup \{\epsilon\}) \to P(Q)$ là hàm chuyển trạng thái, mỗi cặp (q, a) hoặc (q, ϵ) có thể dẫn tới nhiều trạng thái.
- $q_0 \in Q$ là trạng thái bắt đầu.

- $F \subseteq Q$ là tập các trạng thái kết thúc (accepting states).

Nguyên tắc hoạt động:

- Tại một trạng thái q, NFAϵ có thể:
  - Chuyển sang các trạng thái trong $\delta(q, a)$ nếu đọc ký hiệu $a \in \Sigma$.
  - Chuyển sang các trạng thái trong $\delta(q, \epsilon)$ mà không tiêu thụ ký hiệu đầu vào (epsilon-transition).
- Một chuỗi $w \in \Sigma^*$ được chấp nhận nếu tồn tại ít nhất một đường đi từ q_0 đến trạng thái $q_f \in F$ theo các ký hiệu trong w và có thể xen kẽ các chuyển ϵ.

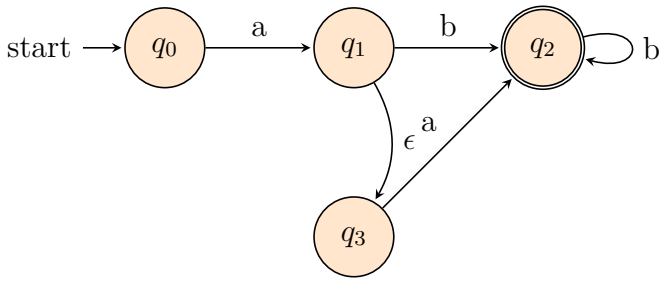
Hàm mở rộng: Hàm mở rộng $\hat{\delta} : Q \times \Sigma^* \to P(Q)$ được định nghĩa bằng:

$$\hat{\delta}(q, \epsilon) = \epsilon\text{-closure}(q)$$

$$\hat{\delta}(q, xa) = \bigcup_{p \in \hat{\delta}(q,x)} \epsilon\text{-closure}(\delta(p, a)), \quad x \in \Sigma^*, a \in \Sigma$$

Trong đó, **ϵ-closure** của một trạng thái q, ký hiệu ϵ-closure(q), là tập tất cả các trạng thái có thể đạt được từ q chỉ bằng các chuyển ϵ (bao gồm chính q).

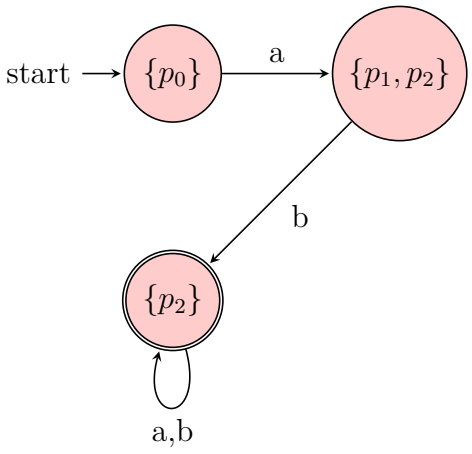
Ví dụ trực quan:



Ghi chú:

- Trạng thái q_1 có thể đi tới q_3 mà không cần đọc ký hiệu (ϵ-transition).
- Chuỗi aab có thể được chấp nhận nhờ việc di chuyển qua ϵ từ q_1 sang q_3.
- NFAϵ có thể được chuyển đổi sang DFA (không epsilon) bằng thuật toán epsilon-closure và subset construction.

4.2 Chuyển đổi NFA sang DFA



Thuật toán xây dựng tập con (subset construction) Cho NFA $N = (Q_N, \Sigma, \delta_N, q_0, F_N)$, xây dựng DFA $D = (Q_D, \Sigma, \delta_D, q_0^D, F_D)$:

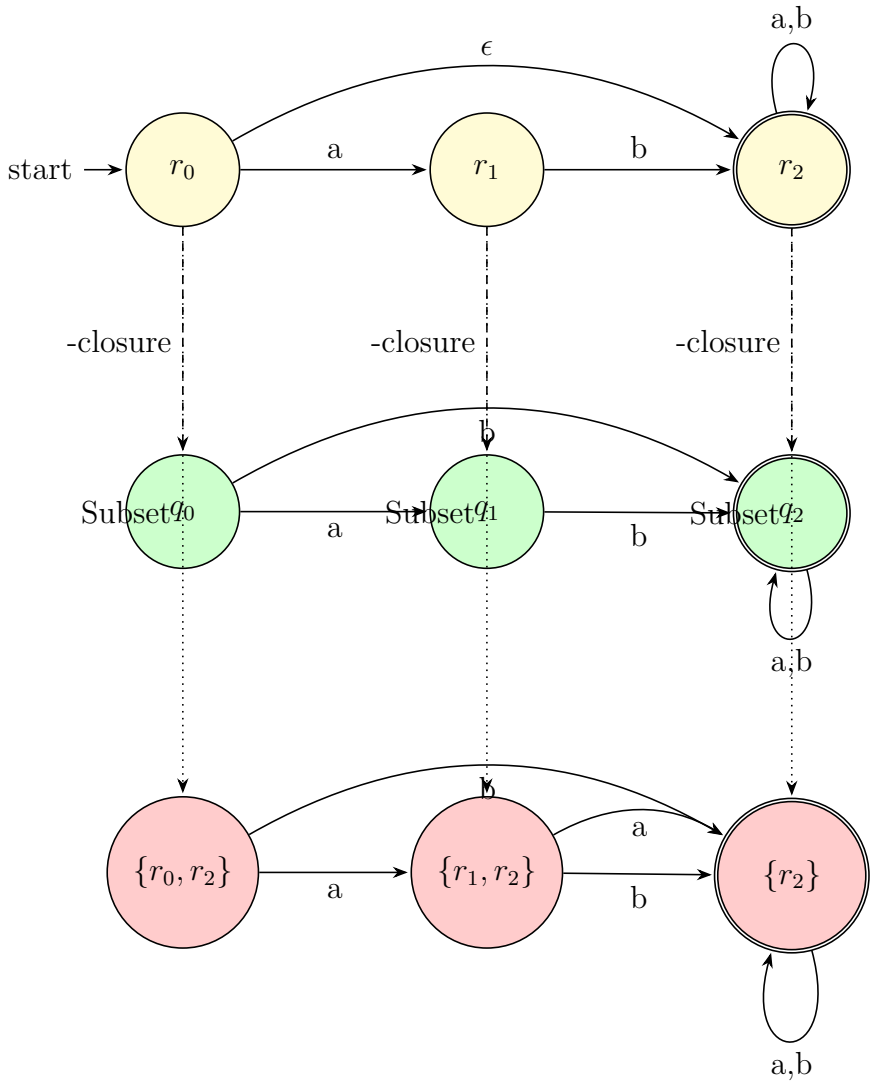
- $Q_D \subseteq P(Q_N)$, mỗi trạng thái DFA là một tập con các trạng thái NFA.
- Trạng thái bắt đầu $q_0^D = \{q_0\}$.
- Hàm chuyển DFA:

$$\delta_D(S, a) = \bigcup_{q \in S} \delta_N(q, a), \quad S \subseteq Q_N, a \in \Sigma$$

- Trạng thái kết thúc: $F_D = \{S \subseteq Q_N \mid S \cap F_N \neq \emptyset\}$.

- Mỗi trạng thái DFA đại diện cho **tập hợp trạng thái NFA có thể đến**. - Quá trình này đảm bảo DFA **chấp nhận cùng ngôn ngữ** với NFA ban đầu.

4.3 Chuyển NFAϵ sang NFA hoặc DFA



4.3.1 Epsilon-closure (E-closure)

Định nghĩa: Epsilon-closure của một trạng thái q trong NFAϵ, ký hiệu $E(q)$, là tập tất cả các trạng thái mà NFAϵ có thể tới từ q bằng **một hoặc nhiều phép chuyển** ϵ, bao gồm chính q:

$$E(q) = \{p \in Q \mid q \xrightarrow{\epsilon*} p\}$$

Ví dụ: nếu $q_0 \xrightarrow{\epsilon} q_1 \xrightarrow{\epsilon} q_2$, thì $E(q_0) = \{q_0, q_1, q_2\}$.

Ý nghĩa: - Epsilon-closure giúp loại bỏ các chuyển epsilon khi chuyển NFAϵ sang NFA thông thường. - Sau khi loại bỏ epsilon, NFA chỉ còn các chuyển trạng thái theo ký hiệu trong Σ.

4.3.2 Thuật toán chuyển NFAϵ sang NFA

1. **Tính epsilon-closure cho mỗi trạng thái:**

$$\forall q \in Q, \quad E(q) = \text{tập tất cả các trạng thái có thể tới từ } q \text{ bằng epsilon}$$

2. **Xây dựng hàm chuyển mới δ':** Với mỗi ký tự $a \in \Sigma$ và trạng thái $q \in Q$, tập trạng thái kế tiếp được xác định bằng:

$$\delta'(q, a) = \bigcup_{p \in E(q)} \delta(p, a)$$

   - Ở đây, $\delta(p, a)$ là các trạng thái NFA có thể tới từ p với ký tự a. - δ' loại bỏ epsilon, chỉ còn chuyển theo ký tự trong Σ.

3. **Xác định trạng thái kết thúc:** Một trạng thái q trong NFA mới là trạng thái kết thúc nếu epsilon-closure của nó chứa ít nhất một trạng thái kết thúc trong NFAϵ:

$$q \in F' \iff E(q) \cap F \neq \emptyset$$

4.3.3 Thuật toán chuyển NFAϵ sang DFA

Quy trình tổng quát gồm 2 bước chính:

1. **Bước 1: Chuyển NFAϵ sang NFA** - Dùng epsilon-closure để loại bỏ các chuyển epsilon như hướng dẫn ở trên. - Kết quả là NFA thông thường chỉ có các chuyển theo ký hiệu trong Σ.

2. **Bước 2: Chuyển NFA sang DFA (Subset Construction)**

   (a) Mỗi trạng thái DFA là một tập con các trạng thái NFA.

   (b) Trạng thái bắt đầu DFA: $\{q_0\}$ hoặc epsilon-closure của q_0 nếu xuất phát từ NFAϵ.

   (c) Hàm chuyển DFA: với tập con $S \subseteq Q$ và ký tự $a \in \Sigma$, tập trạng thái kế tiếp là:

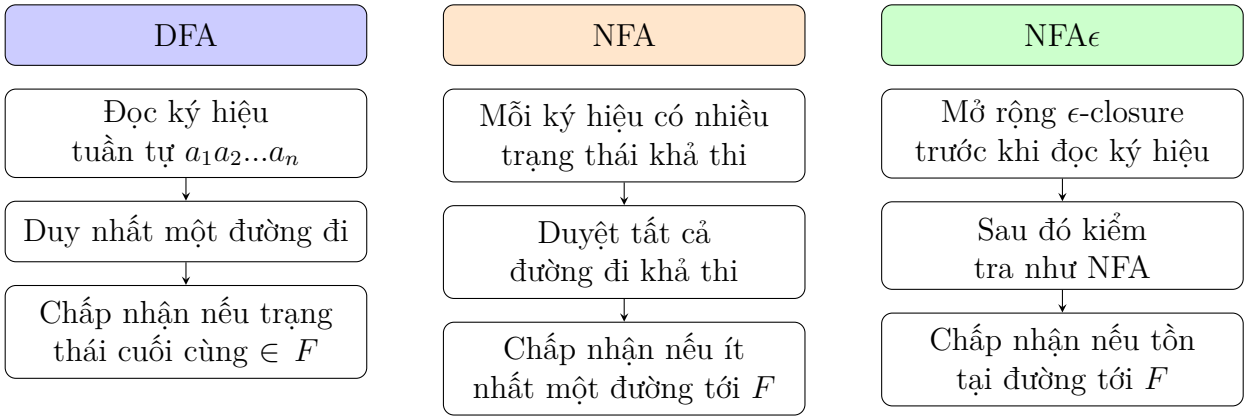
$$\delta_D(S, a) = \bigcup_{q \in S} \delta'(q, a)$$

   (d) Trạng thái kết thúc DFA: mọi tập con chứa ít nhất một trạng thái kết thúc của NFA.

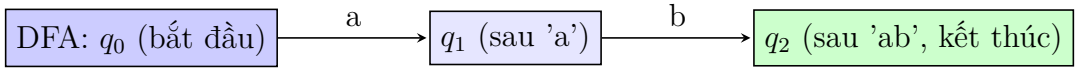
Lưu ý:

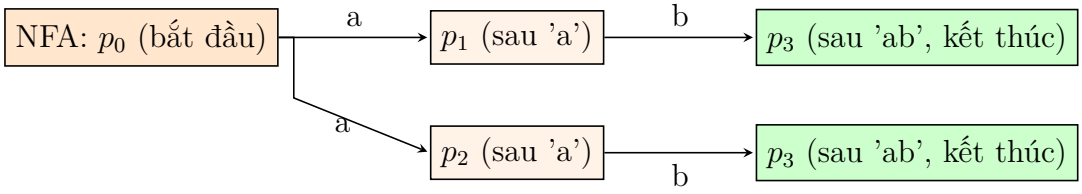
- Subset construction có thể tạo ra tối đa $2^{|Q|}$ trạng thái DFA, nhưng thường ít hơn nhiều vì nhiều tập con không bao giờ xuất hiện.
- Kết quả là một DFA chấp nhận cùng ngôn ngữ với NFAϵ ban đầu.

4.4 Kiểm tra chuỗi trong DFA, NFA, NFAϵ



Ví dụ kiểm tra chuỗi $w = ab$ trên DFA, NFA, NFAϵ







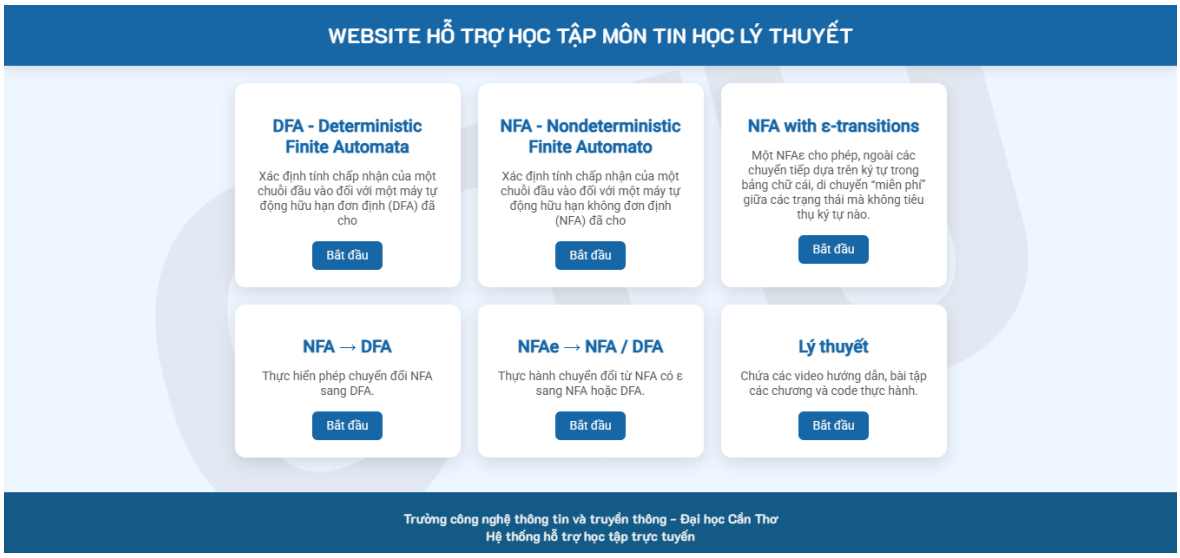
Chương 5

CHỨC NĂNG CỦA HỆ THỐNG

Chương này trình bày chi tiết các chức năng chính của hệ thống website hỗ trợ học tập môn *Tin học lý thuyết*. Hệ thống được xây dựng nhằm giúp sinh viên kiểm tra, mô phỏng và thực hành các bài toán liên quan đến ôtômát hữu hạn, bao gồm DFA, NFA và NFAϵ, đồng thời hỗ trợ chuyển đổi giữa các mô hình và cung cấp lý thuyết kèm bài tập.

5.1 Tổng quan

Website được thiết kế với giao diện trực quan, dễ sử dụng, cung cấp các chức năng chính như kiểm tra chuỗi, chuyển đổi mô hình, và học tập lý thuyết. Mỗi chức năng được mô tả chi tiết dưới đây.



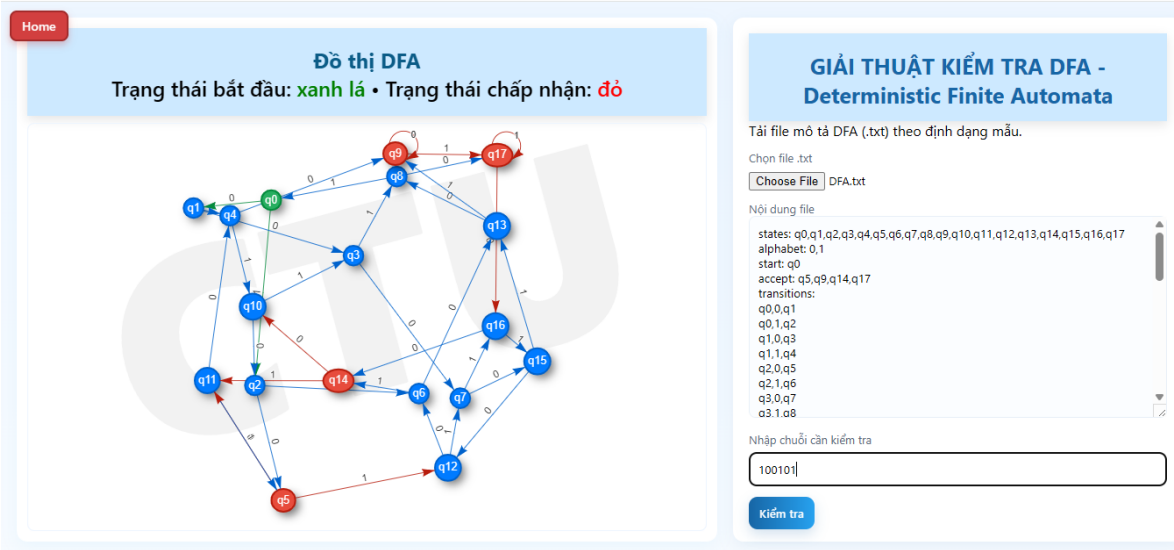
Hình 5.1: Giao diện chính của website

5.2 Chức năng kiểm tra DFA

- Cho phép người dùng **nhập một DFA** bao gồm:
  - Tập trạng thái Q.
  - Bảng chuyển trạng thái δ.

- Trạng thái bắt đầu q_0.
- Tập trạng thái kết thúc F.
- Bảng ký hiệu đầu vào Σ.

- Hỗ trợ **nhập chuỗi đầu vào** và kiểm tra xem chuỗi có được DFA chấp nhận hay không.
- Quá trình kiểm tra được **hiển thị từng bước**, làm nổi bật các trạng thái đang đi qua.
- Cung cấp **kết quả cuối cùng**: chuỗi được chấp nhận hay bị từ chối.



Hình 5.2: Giao diện trang kiểm tra DFA

Hàm `simulateDFA(dfa, str)` dùng để mô phỏng cách một DFA xử lý một chuỗi ký tự. Nó bắt đầu bằng việc lấy trạng thái bắt đầu và khởi tạo các mảng lưu đường đi (`path`) và chi tiết từng bước (`details`), đồng thời tạo bảng chuyển δ để hiển thị. Nếu chuỗi rỗng, hàm kiểm tra ngay trạng thái bắt đầu có thuộc tập trạng thái chấp nhận hay không và trả kết quả. Nếu chuỗi không rỗng, hàm sẽ duyệt từng ký tự: với mỗi ký tự, nó tìm trạng thái tiếp theo theo bảng chuyển, lưu lại trạng thái hiện tại vào đường đi và chi tiết δ vào mảng `details`. Nếu gặp ký hiệu không có chuyển hợp lệ, hàm dừng và báo lỗi. Khi hết chuỗi, hàm kiểm tra trạng thái cuối cùng có thuộc tập `accept` hay không, tạo kết luận dạng LaTeX và trả về toàn bộ thông tin, bao gồm: kết quả chấp nhận/chối, đường đi các trạng thái, chi tiết từng bước và bảng δ, phục vụ cho việc hiển thị trực quan trên giao diện.

```
function simulateDFA(dfa, str) {
  const start = dfa.start;
  const details = [];
  const path = [start];
  const deltaTable = buildDeltaTableHTML(dfa);

  if (!str || str.length === 0) {
    details.push(wrapInline('\\delta(' + stateToTex(start) +
    ', \\epsilon) = ' + stateToTex(start)));
    const accepted0 = dfa.accept.has(start);
    const conclusion = accepted0
      ? wrapDisplay('\\delta(' + stateToTex(start) + ', \\epsilon) = '
      + stateToTex(start) + ' \\in F \\Rightarrow \\text{Chuỗi rỗng được chấp nhận.}')
      : wrapDisplay('\\delta(' + stateToTex(start) + ', \\epsilon) = ' + stateToTex(start) +
      ' \\notin F \\Rightarrow \\text{Chuỗi rỗng bị từ chối.}');
    return { ok: true, accepted: accepted0, path, details, deltaTable, conclusion };
  }
```

```
  let cur = start;
  for (let i = 0; i < str.length; i++) {
    const prefix = str.slice(0, i + 1);
    const nested = makeNestedShort(start, prefix, dfa);
    details.push(nested);

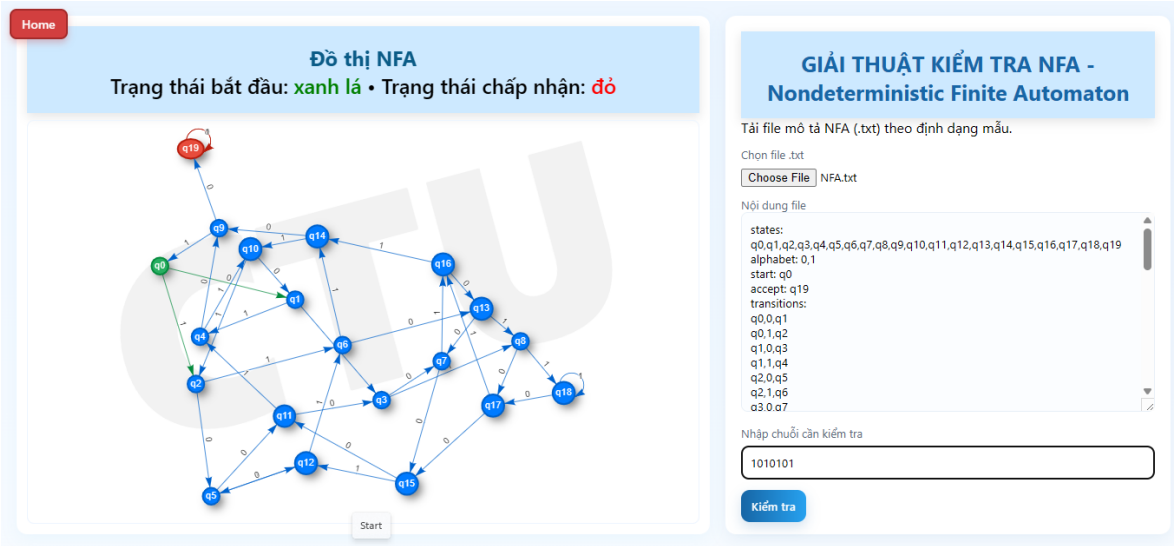
    const sym = str[i];
    const trans = (dfa.transitions[cur] && dfa.transitions[cur][sym])
                  !== undefined ? dfa.transitions[cur][sym] : null;
    if (trans === null) {
      details.push(wrapInline('\\text{(Dừng: không có chuyển từ '
        + stateToTex(cur) + ' với ký hiệu }' + sym + ')'));
      return { ok: false, msg: `Không có chuyển từ ${cur} với ký hiệu '${sym}'`,
       path, details, deltaTable };
    }
    cur = trans;
    path.push(cur);
  }
```

```
  const accepted = dfa.accept.has(cur);
  const conclusion = accepted
    ? wrapDisplay('\\delta(' + stateToTex(start) + ', ' + str + ') = '
        + stateToTex(cur) + ' \\in F \\Rightarrow \\text{Chuỗi được chấp nhận.}')
    : wrapDisplay('\\delta(' + stateToTex(start) + ', ' + str + ') = '
        + stateToTex(cur) + ' \\notin F \\Rightarrow \\text{Chuỗi bị từ chối.}');
  return { ok: true, accepted, path, details, deltaTable, conclusion };
}
```

Hình 5.3: Hàm simulateDFA kiểm tra chuỗi thõa mãn DFA

5.3 Chức năng kiểm tra NFA

- Cho phép người dùng nhập một NFA với các thông tin tương tự DFA.
- Hỗ trợ kiểm tra chuỗi đầu vào **dựa trên tất cả các đường đi khả thi**.
- Giao diện hiển thị:
  - Tất cả các đường đi đang được kiểm tra.
  - Các trạng thái cuối cùng nếu chuỗi được chấp nhận.
- Giúp sinh viên **quan sát đặc trưng phi định hướng của NFA**.



Hình 5.4: Giao diện trang kiểm tra NFA

Hàm `simulateNFA(nfa, str)` dùng để mô phỏng cách một **NFA (Non-deterministic Finite Automaton)** xử lý một chuỗi ký tự. Nó bắt đầu bằng việc khởi tạo tập các trạng thái hiện tại (`currentStates`) chỉ chứa trạng thái bắt đầu, đồng thời tạo bảng chuyển δ (`deltaTable`) để hiển thị. Nếu chuỗi rỗng, hàm kiểm tra xem trạng thái bắt đầu có thuộc tập trạng thái chấp nhận hay không và tạo kết luận. Nếu chuỗi không rỗng, hàm duyệt từng ký tự trong chuỗi: với mỗi ký tự, nó xét tất cả các trạng thái hiện tại, tìm tất cả trạng thái đích mà NFA có thể chuyển tới, lưu tập các trạng thái này làm trạng thái hiện tại mới và ghi chi tiết bước vào mảng `details` dưới dạng biểu thức LaTeX. Nếu tại bất kỳ bước nào không còn trạng thái nào để chuyển, hàm dừng và báo lỗi. Khi hết chuỗi, hàm kiểm tra xem tập trạng thái cuối cùng có giao với tập trạng thái chấp nhận hay không để xác định chuỗi được chấp nhận hay bị từ chối, đồng thời trả về toàn bộ thông tin: kết quả, đường đi các tập trạng thái, chi tiết từng bước và bảng δ, phục vụ hiển thị trực quan trên giao diện.

```
function simulateNFA(nfa, str) {
  const deltaTable = buildDeltaTableHTML(nfa);
  let currentStates = new Set([nfa.start]);
  const details = [];
  const path = [new Set(currentStates)];

  function statesToTex(states) {
    if (states.size === 0) return '\\emptyset';
    return '\\{' + [...states].map(s => s.replace(/_/g, '\\_')).join(', ') + '\\}';
  }
  if (!str || str.length === 0) {
    const accepted0 = [...currentStates].some(s => nfa.accept.has(s));
    const conclusion = accepted0
      ? wrapDisplay('\\delta(' + nfa.start + ', \\epsilon) = ' +
        statesToTex(currentStates) +
        ' \\cap F \\neq \\emptyset \\Rightarrow \\text{Chuỗi rỗng được chấp nhận.}')
      : wrapDisplay('\\delta(' + nfa.start + ', \\epsilon) = ' +
        statesToTex(currentStates) +
        ' \\cap F = \\emptyset \\Rightarrow \\text{Chuỗi rỗng bị từ chối.}');
    details.push(wrapInline('\\delta(' + nfa.start + ', \\epsilon) = ' +
      statesToTex(currentStates)));
    return { ok: true, accepted: accepted0, path, details, deltaTable, conclusion };
```

```
  for (let i = 0; i < str.length; i++) {
    const prefix = str.slice(0, i + 1);
    const sym = str[i];
    const nextStates = new Set();

    for (const s of currentStates) {
      const dests = nfa.transitions[s]?.[sym];
      if (dests) for (const d of dests) nextStates.add(d);
    }

    const texPrev = statesToTex(currentStates);
    const texNext = statesToTex(nextStates);

    const nestedExpr = '\\delta(' + texPrev + ', ' + sym + ') = ' + texNext;
    details.push(wrapInline(nestedExpr));

    if (nextStates.size === 0) {
      details.push(wrapInline('\\text{Dừng: không có chuyển từ } ' +
        texPrev + ' \\text{ với ký hiệu } ' + sym));
      return { ok: false, msg: `Không có chuyển từ ${[...currentStates].join(', ')} với '${sym}
      path, details, deltaTable };
    }
```

```
    currentStates = nextStates;
    path.push(new Set(currentStates));
  }

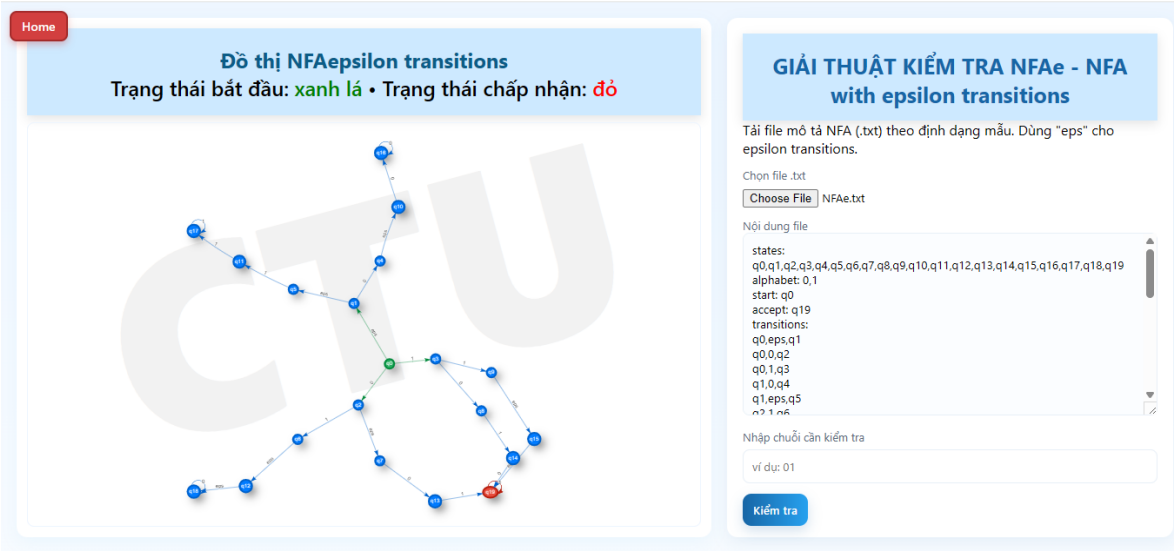
  const accepted = [...currentStates].some(s => nfa.accept.has(s));
  const conclusion = accepted
    ? wrapDisplay('\\delta(' + nfa.start + ', ' + str + ') = ' +
      statesToTex(currentStates) +
      ' \\cap F \\neq \\emptyset \\Rightarrow \\text{Chuỗi được chấp nhận.}')
    : wrapDisplay('\\delta(' + nfa.start + ', ' + str + ') = ' +
      statesToTex(currentStates) +
      ' \\cap F = \\emptyset \\Rightarrow \\text{Chuỗi bị từ chối.}');

  return { ok: true, accepted, path, details, deltaTable, conclusion };
}
```

Hình 5.5: Hàm simulateNFA kiểm tra chuỗi thõa mãn NFA

5.4 Chức năng kiểm tra NFAϵ

- Nhập NFAϵ với các mũi tên ϵ.
- Tự động tính **epsilon-closure** của từng trạng thái.
- Cho phép kiểm tra chuỗi đầu vào tương tự NFA.
- Giao diện trực quan hiển thị:
  - Các bước epsilon-closure.
  - Đường đi khả thi theo chuỗi đầu vào.



Hình 5.6: Giao diện trang kiểm tra NFAe

Hàm `simulateNFAe(nfa, str)` được sử dụng để mô phỏng việc NFA xử lý một chuỗi ký tự có thể chứa epsilon. Khi bắt đầu, hàm lấy trạng thái bắt đầu của NFA và khởi tạo các tập trạng thái hiện tại (`currentStates`), mảng lưu đường đi (`path`) và chi tiết từng bước (`details`). Hàm `statesToTex` được định nghĩa để chuyển các tập trạng thái thành biểu diễn LaTeX, phục vụ hiển thị trực quan.

Nếu chuỗi nhập vào rỗng, hàm ngay lập tức kiểm tra xem trạng thái bắt đầu có thuộc tập trạng thái chấp nhận hay không và tạo kết luận tương ứng, đồng thời ghi chi tiết vào mảng `details`.

Nếu chuỗi không rỗng, hàm duyệt từng ký tự trong chuỗi. Với mỗi ký tự, hàm tạo một tập trạng thái tiếp theo (`nextStates`) bằng cách duyệt các trạng thái hiện tại và tìm các chuyển hợp lệ theo bảng chuyển δ của NFA. Sau đó, biểu diễn các bước chuyển được ghi vào `details`. Nếu tại bước nào không có chuyển hợp lệ, hàm dừng và trả về thông báo lỗi cùng chi tiết các bước đã thực hiện.

Khi hết chuỗi, hàm kiểm tra xem tập trạng thái cuối cùng có chứa ít nhất một trạng thái chấp nhận hay không, tạo kết luận về việc chuỗi được chấp nhận hay bị từ chối, và trả về một đối tượng gồm: trạng thái chấp nhận/chối, đường đi qua các trạng thái, chi tiết từng bước và bảng δ, phục vụ cho việc hiển thị trực quan trên giao diện. Như vậy, hàm này cung cấp cơ chế mô phỏng NFA một cách chi tiết, đồng thời hỗ trợ trực quan hóa các bước xử lý chuỗi với epsilon.

```
function simulateNFAe(nfa, str) {
  const deltaTable = buildDeltaTableHTML(nfa);
  let currentStates = new Set([nfa.start]);
  const details = [];
  const path = [new Set(currentStates)];

  function statesToTex(states) {
    if (states.size === 0) return '\\emptyset';
    return '\\{' + [...states].map(s => s.replace(/_/g, '\\_')).join(', ') + '\\}';
  }


  if (!str || str.length === 0) {
    const accepted0 = [...currentStates].some(s => nfa.accept.has(s));
    const conclusion = accepted0
      ? wrapDisplay('\\delta(' + nfa.start + ', \\epsilon) = '
        + statesToTex(currentStates) +
        ' \\cap F \\neq \\emptyset \\Rightarrow \\text{Chuỗi rỗng được chấp nhận.}')
      : wrapDisplay('\\delta(' + nfa.start + ', \\epsilon) = '
        + statesToTex(currentStates) +
        ' \\cap F = \\emptyset \\Rightarrow \\text{Chuỗi rỗng bị từ chối.}');
    details.push(wrapInline('\\delta(' + nfa.start +
```

```
      ', \\epsilon) = ' + statesToTex(currentStates)));
    return { ok: true, accepted: accepted0, path, details, deltaTable, conclusion };
  }

  for (let i = 0; i < str.length; i++) {
    const prefix = str.slice(0, i + 1);
    const sym = str[i];
    const nextStates = new Set();

    for (const s of currentStates) {
      const dests = nfa.transitions[s]?.[sym];
      if (dests) for (const d of dests) nextStates.add(d);
    }

    const texPrev = statesToTex(currentStates);
    const texNext = statesToTex(nextStates);

    const nestedExpr = '\\delta(' + texPrev + ', ' + sym + ') = ' + texNext;
    details.push(wrapInline(nestedExpr));

    if (nextStates.size === 0) {
      details.push(wrapInline('\\text{Dừng: không có chuyển từ } '
```

```
        + texPrev + ' \\text{ với ký hiệu } ' + sym));
      return { ok: false, msg: `Không có chuyển từ ${[...currentStates].join(', ')}
      với '${sym}'`, path, details, deltaTable };
    }

    currentStates = nextStates;
    path.push(new Set(currentStates));
  }

  const accepted = [...currentStates].some(s => nfa.accept.has(s));
  const conclusion = accepted
    ? wrapDisplay('\\delta(' + nfa.start + ', ' + str + ') = '
      + statesToTex(currentStates) +
      ' \\cap F \\neq \\emptyset \\Rightarrow \\text{Chuỗi được chấp nhận.}')
    : wrapDisplay('\\delta(' + nfa.start + ', ' + str + ') = '
      + statesToTex(currentStates) +
      ' \\cap F = \\emptyset \\Rightarrow \\text{Chuỗi bị từ chối.}');

  return { ok: true, accepted, path, details, deltaTable, conclusion };
}
```

Hình 5.7: Hàm simulateNFAe kiểm tra chuỗi thõa mãn NFAe

5.5 Chức năng chuyển NFA sang DFA

- Sử dụng **thuật toán xây dựng tập con (subset construction)** để chuyển đổi.
- Mỗi trạng thái DFA tương ứng với một tập con trạng thái NFA.
- Giao diện hiển thị:
  - Bảng chuyển trạng thái DFA.
  - Sơ đồ trạng thái trực quan.
  - Liên kết từng trạng thái DFA với tập con NFA tương ứng.



Hình 5.8: Giao diện chức năng chuyển NFA sang DFA

Hàm `convertNFAtoDFA(nfa, callback)` thực hiện việc chuyển đổi một NFA sang DFA. Trước tiên, hàm tạo tất cả các tập con có thể của tập trạng thái NFA, mỗi tập con sẽ đại diện cho một trạng thái tiềm năng của DFA. Mỗi tập con được gán một tên duy nhất dựa trên ký tự chữ cái (A, B, C, ...). Sau đó, hàm xây dựng bảng chuyển đổi của DFA (`dfaTrans`) bằng cách duyệt qua từng tập con và từng ký hiệu trong bảng chữ cái (trừ ký hiệu epsilon). Với mỗi trạng thái NFA trong tập con, nếu tồn tại chuyển với ký hiệu hiện tại, hàm sẽ tính epsilon-closure của trạng thái đích và gộp vào tập trạng thái tiếp theo của DFA. Nếu tập trạng thái tiếp theo không rỗng, nó sẽ được ghi vào bảng chuyển và gọi hàm `callback` để lưu chi tiết bước chuyển đổi, phục vụ hiển thị trực quan. Trạng thái bắt đầu của DFA được xác định là epsilon-closure của trạng thái bắt đầu NFA, và tập trạng thái chấp nhận của DFA gồm tất cả các tập con chứa ít nhất một trạng thái chấp nhận của NFA. Cuối cùng, hàm trả về đối tượng DFA bao gồm tập trạng thái, bảng chữ cái (loại trừ epsilon), trạng thái bắt đầu, tập trạng thái chấp nhận và bảng chuyển. Như vậy, hàm này cung cấp cơ chế chuyển đổi toàn diện từ NFA sang DFA, đồng thời cho phép quan sát chi tiết quá trình xây dựng thông qua callback.

```
function convertNFAtoDFA(nfa, callback) {
    const allStates = [...nfa.states];
    const subsets = [];
    for (let mask = 0; mask < (1 << allStates.length); mask++) {
        const subset = new Set();
        for (let i = 0; i < allStates.length; i++) {
            if (mask & (1 << i)) subset.add(allStates[i]);
        }
        subsets.push(subset);
    }

    const stateNames = {};
    let nameCharCode = 65;
    subsets.forEach(subset => {
        const key = [...subset].sort().join(',');
        stateNames[key] = String.fromCharCode(nameCharCode++);
    });
```

```
    const dfaTrans = {};
    subsets.forEach(subset => {
        const key = [...subset].sort().join(',');
        const dfaStateName = stateNames[key];
        dfaTrans[dfaStateName] = {};

        nfa.alphabet.forEach(sym => {
            if (sym === 'ε') return;
            const nextSet = new Set();
            subset.forEach(s => {
                if (nfa.transitions[s] && nfa.transitions[s][sym]) {
                    nfa.transitions[s][sym].forEach(t => {
                        epsilonClosure(nfa, new Set([t])).forEach(x => nextSet.add(x));
                    });
                }
            });
            if (nextSet.size) {
                const nextKey = [...nextSet].sort().join(',');
                dfaTrans[dfaStateName][sym] = stateNames[nextKey];
                if (callback) callback({ current: subset, sym, next: nextSet });
            }
        });
```

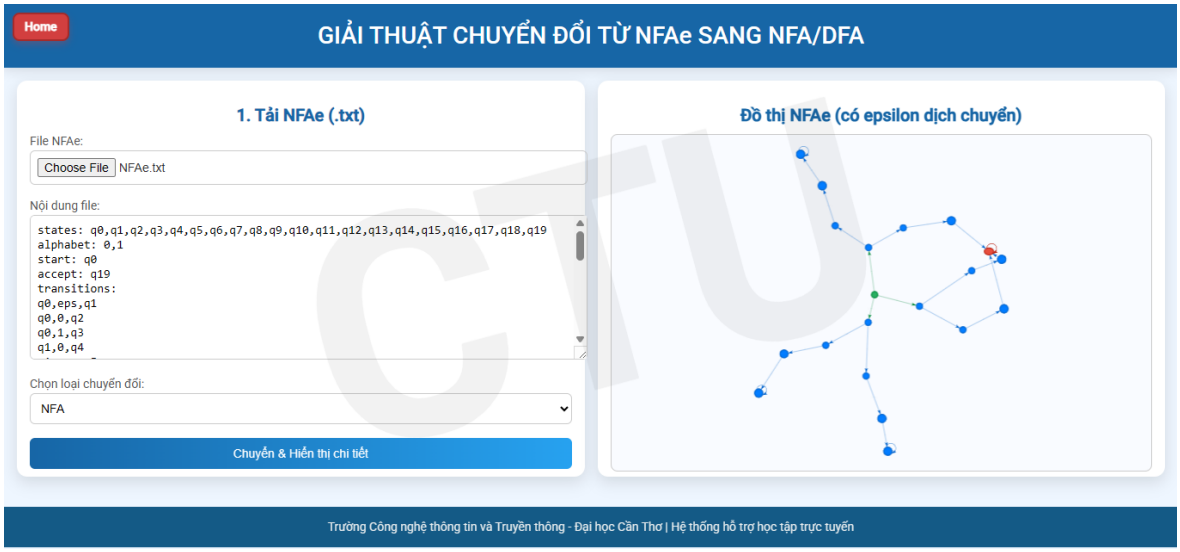
```
    const startClosure = epsilonClosure(nfa, new Set([nfa.start]));
    const startKey = [...startClosure].sort().join(',');
    const accept = new Set();
    subsets.forEach(subset => {
        if ([...subset].some(s => nfa.accept.has(s))) {
            const key = [...subset].sort().join(',');
            accept.add(stateNames[key]);
        }
    });

    return {
        states: new Set(Object.values(stateNames)),
        alphabet: new Set([...nfa.alphabet].filter(a => a !== 'ε')),
        start: stateNames[startKey],
        accept,
        transitions: dfaTrans
    };
}
```

Hình 5.9: Hàm convertNFAtoDFA chuyển NFA sang DFA

5.6 Chức năng chuyển NFAϵ sang DFA hoặc NFA

- Hỗ trợ hai dạng chuyển đổi:
  1. NFAϵ $\rightarrow$ NFA: loại bỏ các mũi tên ϵ thông qua epsilon-closure.
  2. NFAϵ $\rightarrow$ DFA: kết hợp epsilon-closure và subset construction.
- Giao diện trực quan:
  - Hiển thị sơ đồ NFAϵ ban đầu.
  - Hiển thị NFA hoặc DFA kết quả, có mũi tên nối từ trạng thái cũ sang trạng thái mới.
  - Hiển thị bảng chuyển trạng thái chi tiết.



Hình 5.10: Giao diện chức năng chuyển NFAe sang NFA hoặc DFA

Hàm `epsilonClosure(nfae, states)` được sử dụng để tính *epsilon-closure* của một tập trạng thái trong NFA có epsilon (NFAϵ). Khi được gọi, hàm bắt đầu với một ngăn xếp (`stack`) chứa tất cả các trạng thái trong tập `states` và một tập `closure` cũng bao gồm các trạng thái này.

Trong vòng lặp `while`, hàm lấy ra từng trạng thái từ ngăn xếp và kiểm tra xem từ trạng thái đó có các chuyển ϵ hay không. Nếu tồn tại, mỗi trạng thái đích của chuyển ϵ sẽ được thêm vào tập `closure` nếu chưa có, đồng thời được đẩy vào ngăn xếp để tiếp tục kiểm tra các chuyển ϵ tiếp theo.

Quá trình này tiếp tục cho đến khi không còn trạng thái nào trong ngăn xếp, đảm bảo rằng tất cả các trạng thái có thể đạt được từ tập trạng thái ban đầu thông qua các chuyển ϵ đều được bao gồm trong `closure`. Cuối cùng, hàm trả về tập `closure` này, cho phép sử dụng trong việc chuyển đổi NFA sang DFA hoặc mô phỏng NFA.

```
function epsilonClosure(nfae, states){
    const stack=[...states], closure=new Set(states);
    while(stack.length){
        const s=stack.pop();
        if(nfae.transitions[s] && nfae.transitions[s]['ε']){
            nfae.transitions[s]['ε'].forEach(t=>{
                if(!closure.has(t)){ closure.add(t); stack.push(t); }
            });
        }
    }
    return closure;
}
```

Hình 5.11: Hàm epsilonClosure

Hàm `convertNFAeToNFA(nfae)` được sử dụng để chuyển một NFA có epsilon (ϵ-NFA) thành một NFA thông thường (không có epsilon). Đầu tiên, hàm tạo một đối tượng `newTrans` để lưu các bảng chuyển mới. Với mỗi trạng thái `s` trong NFA gốc, hàm duyệt qua từng ký hiệu trong bảng chữ cái (ngoại trừ ϵ).

Đối với mỗi ký hiệu, hàm tính *epsilon-closure* của trạng thái `s` (`eps_s`) để xác định tất cả các trạng thái có thể đạt được từ `s` chỉ bằng các chuyển ϵ. Sau đó, với mỗi trạng thái `q` trong `eps_s`, nếu có chuyển ký hiệu hiện tại từ `q` đến trạng thái `r`, hàm tiếp tục lấy *epsilon-closure* của `r` và thêm tất cả các trạng thái này vào tập đích `dstSet`. Nếu tập này không rỗng, nó được lưu trong bảng chuyển `newTrans` cho trạng thái `s` và ký hiệu hiện tại.

Tiếp theo, hàm xác định tập trạng thái chấp nhận mới `newAccept` bằng cách thêm các trạng thái `s` nếu *epsilon-closure* của `s` chứa ít nhất một trạng thái chấp nhận của NFA gốc.

Cuối cùng, hàm trả về một NFA mới với các thuộc tính: tập trạng thái (`states`) giữ nguyên, bảng chữ cái loại bỏ ký hiệu ϵ, trạng thái bắt đầu (`start`) giữ nguyên, tập trạng thái chấp nhận mới (`accept`) và bảng chuyển mới (`transitions`), đảm bảo rằng NFA thu được có cùng ngôn ngữ với ϵ-NFA ban đầu nhưng không còn epsilon transitions.

```
function convertNFAeToNFA(nfae){
    const newTrans={};
    nfae.states.forEach(s=>{
        newTrans[s]={};
        nfae.alphabet.forEach(a=>{
            if(a==='ε') return;
            let dstSet=new Set();
            const eps_s=epsilonClosure(nfae,new Set([s]));
            eps_s.forEach(q=>{
                if(nfae.transitions[q] && nfae.transitions[q][a]){
                    nfae.transitions[q][a].forEach(r=>{
                        epsilonClosure(nfae,new Set([r])).forEach(rr=>dstSet.add(rr));
                    });
                }
            });
            if(dstSet.size) newTrans[s][a]=dstSet;
        });
    });
```

```
    const newAccept=new Set();
    nfae.states.forEach(s=>{
        const eps_s=epsilonClosure(nfae,new Set([s]));
        if([...eps_s].some(x=>nfae.accept.has(x))) newAccept.add(s);
    });
    return {states:nfae.states,alphabet:new Set([...nfae.alphabet].filter(a=>'ε'!==a)),
        start:nfae.start,accept:newAccept,transitions:newTrans};
}
```

Hình 5.12: Hàm convertNFAeToNFA chuyển NFAe sang NFA

Hàm `convertNFAtoDFA(nfa, callback)` thực hiện việc chuyển một NFA (có thể có epsilon transitions) sang DFA. Ban đầu, hàm tính *epsilon-closure* của trạng thái bắt đầu NFA để tạo trạng thái bắt đầu của DFA và đánh dấu nó là chưa xử lý. Tên các trạng thái DFA được gán tuần tự từ 'A', 'B', ... dựa trên tập hợp các trạng thái NFA hợp thành.

Trong quá trình lặp, hàm lấy từng trạng thái chưa đánh dấu, duyệt qua tất cả các ký hiệu của bảng chữ cái (ngoại trừ ϵ), và tính tập trạng thái tiếp theo bằng cách áp dụng các chuyển từ trạng thái hiện tại theo ký hiệu và *epsilon-closure*. Nếu tập trạng thái tiếp theo chưa được đặt tên, hàm sẽ tạo tên mới và thêm vào danh sách các trạng thái chưa đánh dấu. Đồng thời, hàm lưu các bước chuyển đổi và gọi `callback` nếu được cung cấp, giúp theo dõi quá trình chuyển đổi.

Sau khi xử lý hết các trạng thái, hàm xây dựng đối tượng DFA cuối cùng với các thuộc tính: tập trạng thái (`states`), bảng chữ cái (`alphabet`) loại bỏ ϵ, trạng thái bắt đầu ('A'), tập trạng thái chấp nhận (`accept`) bao gồm tất cả các trạng thái DFA chứa ít nhất một trạng thái chấp nhận của NFA ban đầu, và bảng chuyển (`transitions`) từ tên trạng thái DFA theo ký hiệu. DFA thu được đảm bảo nhận cùng ngôn ngữ với NFA gốc nhưng không còn epsilon transitions và mỗi ký hiệu dẫn đến đúng một trạng thái tiếp theo.

```
function convertNFAtoDFA(nfa,callback){
    const dfaStates=[], dfaTrans={};
    const startClosure=epsilonClosure(nfa,new Set([nfa.start]));
    const unmarked=[startClosure];
    const stateNames={[[...startClosure].sort().join(',')]:'A'};
    let nameCharCode=66;
    while(unmarked.length){
        const curr=unmarked.shift();
        const currName=stateNames[[...curr].sort().join(',')];
        dfaStates.push(curr);
        dfaTrans[currName]={};
        nfa.alphabet.forEach(sym=>{
            if(sym==='ε') return;
            let nextSet=new Set();
            curr.forEach(s=>{
                if(nfa.transitions[s] && nfa.transitions[s][sym]){
                    nfa.transitions[s][sym].forEach(t=>{
                        epsilonClosure(nfa,new Set([t])).forEach(x=>nextSet.add(x));
                    });
                }
            });
```

```
            if(nextSet.size){
                const key=[...nextSet].sort().join(',');
                if(!stateNames[key]) stateNames[key]=String.fromCharCode(nameCharCode++);
                const nextName=stateNames[key];
                dfaTrans[currName][sym]=new Set([nextName]);
                if(!dfaStates.find(s=>[...s].sort().join(',')===key)) unmarked.push(nextSet);
                if(callback) callback({current:curr,sym:sym,next:nextSet});
            }
        });
    }
    const dfaObj={states:new Set(Object.values(stateNames)),alphabet:new Set([...nfa.alphabet].
    for(const s in dfaTrans){
        dfaObj.transitions[s]={};
        for(const sym in dfaTrans[s]){
            dfaObj.transitions[s][sym]=[...dfaTrans[s][sym]][0];
        }
    }
    Object.keys(stateNames).forEach(key=>{
        const arr=key.split(',');
        if(arr.some(x=>nfa.accept.has(x))) dfaObj.accept.add(stateNames[key]);
    });
    return dfaObj;
```

Hình 5.13: Hàm convertNFAtoDFA chuyển NFAe sang DFA

5.7 Chức năng lý thuyết và bài tập

- Cung cấp tài liệu lý thuyết đầy đủ về:
  - DFA, NFA, NFAϵ.
  - Thuật toán chuyển đổi giữa các mô hình.
  - Epsilon-closure, subset construction.
- Chứa các bài tập tự luyện theo từng chương:
  - Bài tập kiểm tra chuỗi.
  - Bài tập chuyển đổi mô hình.
  - Bài tập xác định tập epsilon-closure và tập con.

- Hệ thống cho phép sinh viên:
  - Nhập đáp án và nhận phản hồi trực tiếp.
  - So sánh kết quả với đáp án chuẩn.
  - Lưu trữ lịch sử bài tập để theo dõi tiến độ.

Chương 6

TỔNG KẾT VÀ ĐÁNH GIÁ HƯỚNG PHÁT TRIỂN

6.1 Ưu điểm của hệ thống

Hệ thống website hỗ trợ học tập môn *Tin học lý thuyết* được xây dựng nhằm cung cấp môi trường học tập trực quan, sinh động và dễ sử dụng cho sinh viên. Một số ưu điểm chính gồm:

- **Giao diện trực quan và sinh động:** Hệ thống hiển thị các trạng thái, mũi tên chuyển đổi, bảng chuyển trạng thái chi tiết và các bước epsilon-closure hay subset construction, giúp sinh viên quan sát trực tiếp quá trình hoạt động của DFA, NFA và NFAϵ.
- **Tích hợp lý thuyết và thực hành:** Phần lý thuyết đi kèm bài tập minh họa giúp sinh viên vừa học lý thuyết vừa thực hành trực tiếp, kiểm tra kết quả và củng cố kiến thức.
- **Kiểm tra chuỗi chi tiết:** Chức năng kiểm tra chuỗi hiển thị từng bước chuyển trạng thái, làm nổi bật trạng thái đang đi qua, giúp sinh viên dễ dàng theo dõi và nắm bắt cơ chế hoạt động của ôtômát.
- **Minh họa thuật toán chuyển đổi:** Các thuật toán chuyển NFA sang DFA và NFAϵ sang NFA/DFA được trình bày trực quan, giúp sinh viên dễ hình dung mối quan hệ giữa các tập trạng thái.
- **Công nghệ hiện đại:** Sử dụng HTML5, CSS3, JavaScript, Bootstrap, Node.js/Servlets và thư viện vẽ đồ thị, giúp hệ thống hoạt động mượt mà, dễ dàng mở rộng và bảo trì.
- **Tùy chọn lưu trữ linh hoạt:** Hệ thống hỗ trợ lưu trữ dữ liệu ôtômát qua LocalStorage hoặc cơ sở dữ liệu MySQL/MongoDB, mang lại trải nghiệm học tập linh hoạt và trực quan.

6.2 Hạn chế của hệ thống

Mặc dù hệ thống đã đạt được mục tiêu cơ bản, vẫn tồn tại một số hạn chế cần khắc phục trong các phiên bản tiếp theo:

- **Hiệu năng với ôtômát lớn:** Khi nhập các NFAϵ phức tạp với nhiều trạng thái và epsilon-closure, hệ thống trên trình duyệt đôi khi chạy chậm do giới hạn tài nguyên.
- **Chưa có tạo bài tập tự động:** Hệ thống chưa hỗ trợ sinh ngẫu nhiên bài tập kiểm tra DFA/NFA/NFAϵ, dẫn đến việc sinh viên phải tự tạo bài tập thủ công.
- **Thiếu thống kê tiến độ học tập:** Chưa có chức năng lưu lịch sử thực hành, theo dõi điểm số hay thời gian thực hành, khiến giảng viên khó đánh giá kết quả học tập sinh viên.
- **Chưa tối ưu cho thiết bị di động:** Giao diện chưa hoàn toàn responsive, chưa tối ưu trải nghiệm trên màn hình nhỏ hoặc cảm ứng.
- **Hạn chế trong chia sẻ dữ liệu:** Hệ thống hiện chỉ lưu trữ cục bộ, chưa có tính năng chia sẻ ôtômát và kết quả thực hành trực tuyến giữa nhiều người dùng.

6.3 Hướng phát triển

Dựa trên các ưu điểm và hạn chế, hệ thống có thể phát triển theo các hướng sau để nâng cao hiệu quả học tập và trải nghiệm người dùng:

- **Tối ưu hóa hiệu năng:** Sử dụng các thuật toán tối ưu, hoặc tích hợp WebAssembly để xử lý các ôtômát lớn, giảm độ trễ khi kiểm tra chuỗi và chuyển đổi trạng thái.
- **Sinh bài tập tự động:** Thêm chức năng tạo ngẫu nhiên bài tập kiểm tra DFA, NFA, NFAϵ theo mức độ khó hoặc dựa trên tiến độ học tập của sinh viên.
- **Thống kê và báo cáo học tập:** Lưu trữ kết quả thực hành, theo dõi tiến độ học tập, số lần kiểm tra và điểm số, giúp giảng viên đánh giá và hướng dẫn sinh viên hiệu quả hơn.
- **Hỗ trợ mô hình nâng cao:** Mở rộng sang các mô hình ôtômát phức tạp hơn như Pushdown Automata (PDA) hoặc Turing Machine, phục vụ nghiên cứu nâng cao.
- **Cải thiện giao diện đa nền tảng:** Thiết kế responsive hoàn toàn, hỗ trợ màn hình nhỏ, cảm ứng và thêm hiệu ứng trực quan sinh động.
- **Lưu trữ và chia sẻ dữ liệu trực tuyến:** Kết nối hệ thống với cơ sở dữ liệu đám mây, cho phép sinh viên lưu trữ, chia sẻ ôtômát và kết quả thực hành.
- **Mở rộng phần lý thuyết và bài tập:** Tích hợp thêm các bài tập tự đánh giá, bài tập theo nhóm hoặc chế độ thi thử trực tuyến, tăng cường tương tác và đánh giá kiến thức.

Kết luận Hệ thống đã đạt được mục tiêu cơ bản trong việc hỗ trợ học tập môn *Tin học lý thuyết*, tạo môi trường học tập trực quan, tương tác và sinh động. Các ưu điểm giúp sinh viên nắm vững kiến thức về ôtômát, trong khi các hạn chế và hướng phát triển đã chỉ ra những điểm cần cải tiến trong các phiên bản tiếp theo, nhằm nâng cao hiệu quả giảng dạy và trải nghiệm người dùng.

Tài liệu tham khảo

[1] L. Brown and M. Zhao. Migrating teaching of automata theory to a digital platform. *South African Journal of Computer Science*, 24(2):1–15, 2024.

[2] T. Nguyen and J. Smith. A web app for teaching finite state automata. *arXiv preprint arXiv:2410.12115*, 2024.